

Inferring network from single-cell data with CausFate

Lan-lab

1/14/2026

Structure learning

Import packages

```
library(Seurat) # Seurat is necessary for singel-cell data
library(foreach)
library(tidyverse)
library(bnlearn)
library(Matrix)
library(space)
library(causfate)
```

Import data

```
data("HMR")
head(HMR)
```

```
##           orig.ident nCount_RNA nFeature_RNA celltype percent.mt      S.Score
## HSPC_002          HSPC   1085.3333         8778    LTHSC           0 -0.12077291
## HSPC_004          HSPC   1221.0767        11352    LMPP           0 -0.16803616
## HSPC_006          HSPC   1431.2583        11891    LMPP           0 -0.01206173
## HSPC_008          HSPC   1514.3543        10355    LMPP           0 -0.16419568
## HSPC_011          HSPC   1516.5897        10977    LMPP           0 -0.19785059
## HSPC_014          HSPC   1470.6718         9533     MPP           0 -0.16047674
## HSPC_015          HSPC    800.7971         9825    LMPP           0 -0.10753757
## HSPC_017          HSPC   1386.0569        10124    LTHSC           0 -0.17785950
## HSPC_018          HSPC   1876.0635        11314     MPP           0 -0.18039289
## HSPC_020          HSPC   1715.7519        10017    LTHSC           0 -0.15426335
##           G2M.Score Phase old.ident
## HSPC_002 -0.09083001    G1      HSPC
## HSPC_004 -0.10391938    G1      HSPC
## HSPC_006 -0.11841703    G1      HSPC
## HSPC_008 -0.12593563    G1      HSPC
## HSPC_011 -0.15114819    G1      HSPC
## HSPC_014 -0.13218429    G1      HSPC
## HSPC_015 -0.08138777    G1      HSPC
## HSPC_017 -0.12033867    G1      HSPC
```

```
## HSPC_018 -0.17387886 G1 HSPC
## HSPC_020 -0.15628029 G1 HSPC
```

```
dim(HMR)
```

```
## [1] 40198 716
```

Find variable genes

Note that `gem2mem` is used to transform the single-cell data to cell type-specific gene profiles.

```
HMR <- FindVariableFeatures(HMR, selection.method = "vst", nfeatures = nrow(HMR))
glist <- VariableFeatures(HMR)

meta <- HMR$celltype
ctypes <- names(table(HMR$celltype))
blood_celltype <- gem2mem(data.frame(HMR@assays$RNA@data), meta, "mean")
```

BMA process

`BNLearning` samples the data and performs the inner BMA process, giving out multiple one-shot DAGs. `combineDAGsmp1` combines those DAGs before pruning.

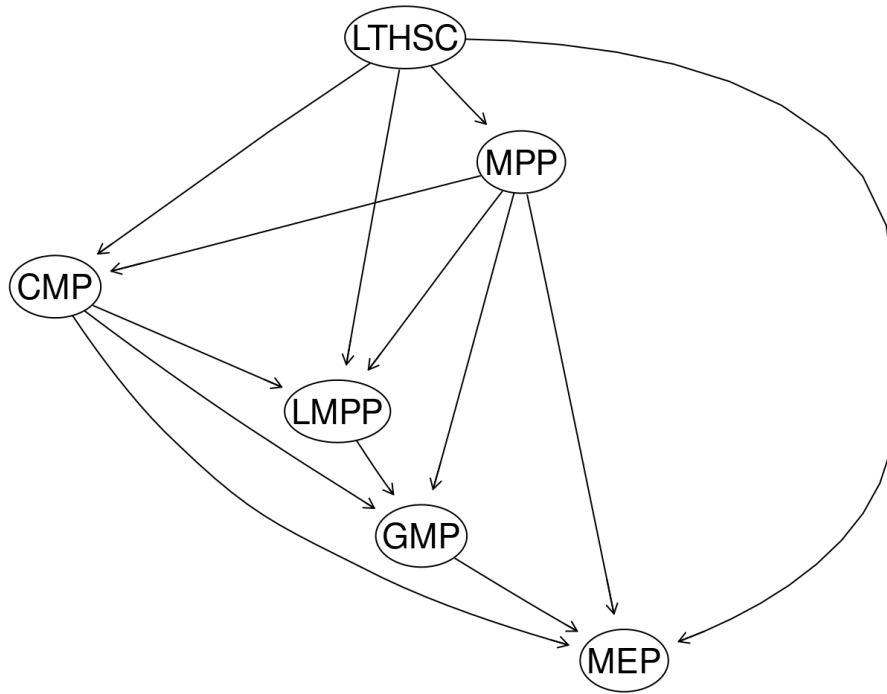
```
dag_smp1 <- BNLearning(HMR[glist[1:5000]],
  frac = 0.2, N_smp1 = 100, params = seq(0.05, 0.3, 0.05),
  root = "LTHSC", mode = "single_cell", ncores = 200,
  dagMethod = "hc", ugMethod = "cmi2ni"
)
net <- combineDAGsmp1(dag_smp1, Emin = 1, Emax = 7, ncores = 64)

# Let us take a glimpse of the structure now
net_mat <- net %>% df2mat()
net_mat
```

```
##      CMP GMP LMPP LTHSC MEP MPP
## CMP      0 290  111      0 278  16
## GMP       5   0    0      0   5    1
## LMPP     57  15    0      0   0 120
## LTHSC     30   0  118      0   4 296
## MEP       3   0    0      0   0    1
## MPP     260  12  176      0  11    0
```

First, very unlikely edges are trimmed if the occurrence is less than 2. Then, `rmCyc` removes cycles, preserving the more possible edges.

```
net <- (net_mat * (net_mat > 2)) %>%
  mat2df() %>%
  rmCyc()
e <- df2bn(net, ctypes, plot = TRUE)
```

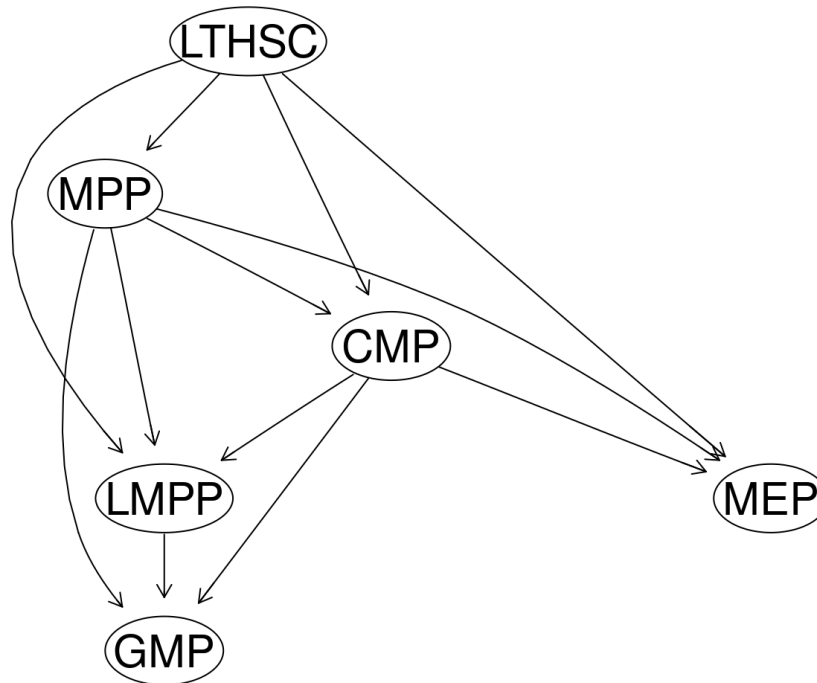


`trimDAG` performs the final local pruning step. Two steps in total are performed for better outcome.

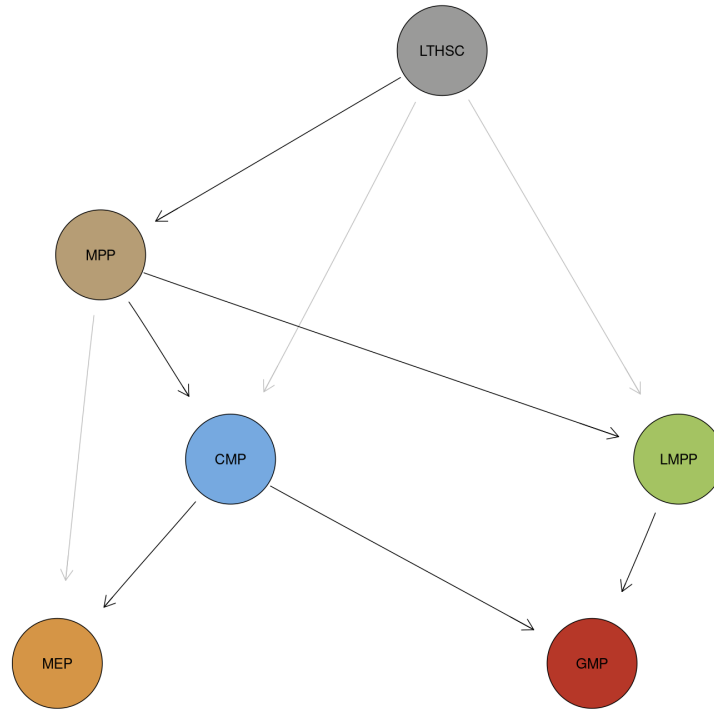
```

tmp <- blood_celltype
tmp2 <- trimDAG(tmp[glist[4000:8000], ], e, min_arc = 2, max_arc = 3, threshold_value = 1)

```



```
tmp2 <- trimDAG(tmp[glist[1:4000], ], tmp2, min_arc = 2, max_arc = 2, threshold_value = 1, plot = TRUE)
dag_struc <- tmp2
# Coloration is omitted for brevity.
```



Calculate gene-wise causal effects

Sampling is used before perturbation. This process could take a while.

```
# Generate indexes for bootstrapping
index <- bootstrap_index(meta, bootstrap_times = 20, ratio = 0.3)
perturbRes <- PerturbResult(dag_struc, HMR, meta, index,
  n_sample = 20,
  mode = "single_cell", ncores = 20
)
effMat <- EffectMatrix(perturbRes, mode = "mean")

ctypes <- c("CMP", "GMP", "LMPP", "LTHSC", "MEP", "MPP")
edgeSet <- setEdges(fromSet = ctypes, toSet = ctypes) # Calculate for all edges
effectScore <- diffScore(effMat, edgeSet)
geneList <- dsRank(effectScore)
head(geneList)
```

```
## [1] "Mpo"          "Ctsg"          "Apoe"          "Plac8"
## [5] "X..not.aligned" "Car1"
```